

KnightFall Chess Engine: Final Report

Introduction / Background

Literature Review

Recent advances in computer chess follow two distinct architectural paradigms.

MCTS-based engines — Silver et al. introduced AlphaZero in 2017, combining deep residual networks with Monte Carlo Tree Search, trained entirely through self-play. These engines require large CNNs on GPU/TPU. The field is now moving toward transformers, as demonstrated by Leela Chess Zero's 2024 work showing grandmaster-level strength at reduced computational cost.

Alpha-Beta search engines with NNUE — Yu Nasu developed NNUE in 2018 for Shogi; Stockfish adopted it in 2020. NNUEs are shallow, sparse networks for fast CPU evaluation. They use sparse encoding and small models updated incrementally as pieces move, making them orders of magnitude faster than MCTS-based approaches.

Our work sits at the intersection: we apply modern routing ideas (MoE, LayerStacks) to the NNUE paradigm, asking whether a learned router can outperform Stockfish's hand-coded piece-count bucketing at the same parameter budget.

Dataset Description

- **Training:** Stockfish master-binpacks + Leela T60/T70/T74/T75/T76 self-play data (``.binpack``) — each record is ``(position, score, WDL)``
- **Evaluation:** Lichess Puzzle DB (CC0 license)
- **Format:** Stockfish ``.binpack`` data — a compact encoding format that is quite expensive to decode and reencode into our model inputs

Problem Definition

Problem

Our goal is to learn a function $f(\text{position}) \rightarrow \text{heuristic value}$ that estimates position strength and replaces Stockfish's evaluation function, running on CPU in nanoseconds. The model outputs in centipawns, where 1 centipawn is 0.01 pawns.

Research Question

Does a learned MoE router beat Stockfish's fixed piece-count bucketing (LayerStacks) at the same parameter budget? More broadly: does the Bitter Lesson hold — does a general learned mechanism outperform a hand-crafted one?

Motivation

Chess is the traditional proving ground for AI. Our goal was to evaluate the strength of MoE related techniques in stockfish.

CHANGES FROM MIDTERM

- Training stack:** Migrated from Rust/Bullet to **PyTorch + Rust** (stockfish-trainer). Bullet only supports a single model type; extending it to LayerStacks and MoE would require custom CUDA kernels. PyTorch gives us flexibility, while our Rust dataloader preserves binary-pack streaming speed.
- Engine target:** Moved from Obsidian to a **custom Stockfish fork** to enable direct ELO measurement and puzzle benchmarking at scale.
- Scope:** Extended from one baseline NNUE to **three architectures** trained under the same recipe — No Stack, LayerStacks, and Mixture-of-Experts — enabling a controlled comparison.

Methods

Data Preprocessing

All models share the same Rust preprocessing pipeline:

- Perspective flip** — positions are encoded from the side-to-move perspective, halving the representational burden and making the model color-agnostic.
- Sparse feature encoding** — two composed feature sets:
 - HalfKAv2_hm:** 24,576 inputs — encodes piece locations relative to the king square, with 32 horizontally-mirrored king buckets and shared virtual weights.
 - Full_Threats:** 60,720 inputs — encodes attack/threat patterns per square, giving the model explicit attack information without search.
- Position filtering** — low-value examples are removed:
 - ``early_fen_skipping = 12`` — skip first 12 plies (opening book noise)
 - ``random_fen_skipping = 10`` — randomly subsample for diversity
 - Filtered/WLD filters — skip resignable and unstable positions (e.g., immediately after captures)

Training Configuration (shared across all models)

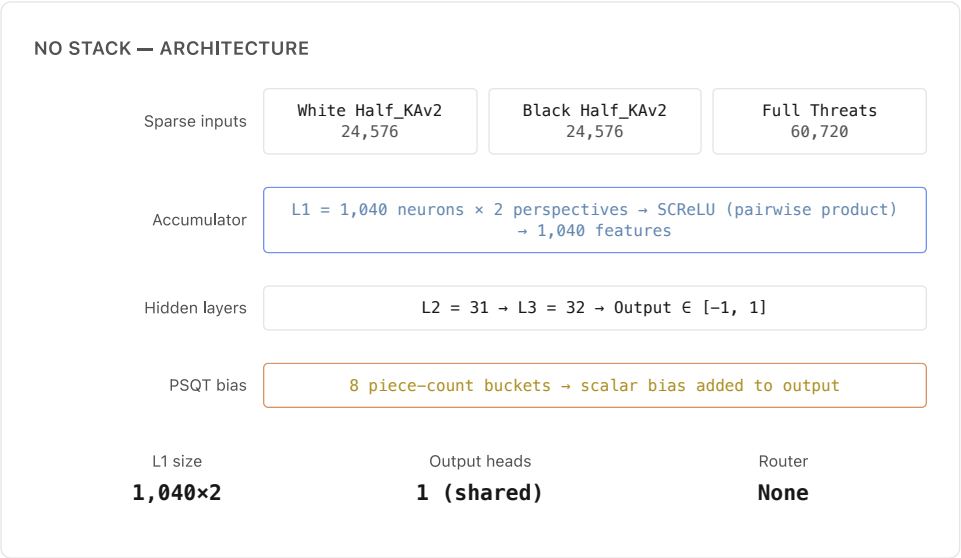
Hyperparameter	Value
Batch size	65,536 positions
Learning rate	4.375×10^{-4} , exponential decay ($\gamma = 0.995$ per epoch)
Loss	Power loss (exponent 2.5); λ annealed $1.0 \rightarrow 0.75$ over training
Optimizer	Ranger22
Hardware	PACE cluster · L40S GPUs
Framework	PyTorch + Rust dataloader
Training data	37.5B positions from Stockfish + Leela self-play

Model 1: No Stack NNUE

The No Stack model is our midterm baseline retrained with the new PyTorch + Rust stack and richer feature encoding.

Architecture:

- Features: composed sparse input of Full_Threats (60,720) + HalfKAv2_hm (24,576), factorized with shared virtual weights
- L1 = 1,040 neurons per perspective, SReLU (pairwise product) = 1,040 features
- L2 = 31, L3 = 32, single output head → [-1, 1]
- 8 parallel PSQT buckets contribute a bias term to the output (selected by piece count)



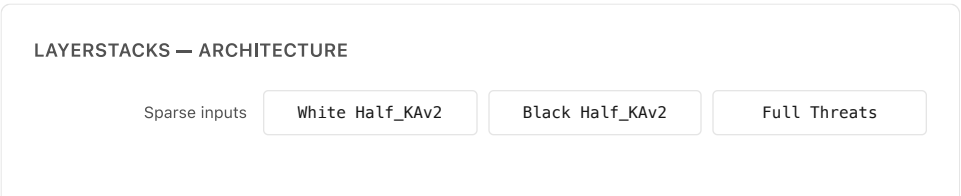
This model serves as the **baseline**: no specialized sub-networks, one shared set of hidden weights for all positions regardless of game phase.

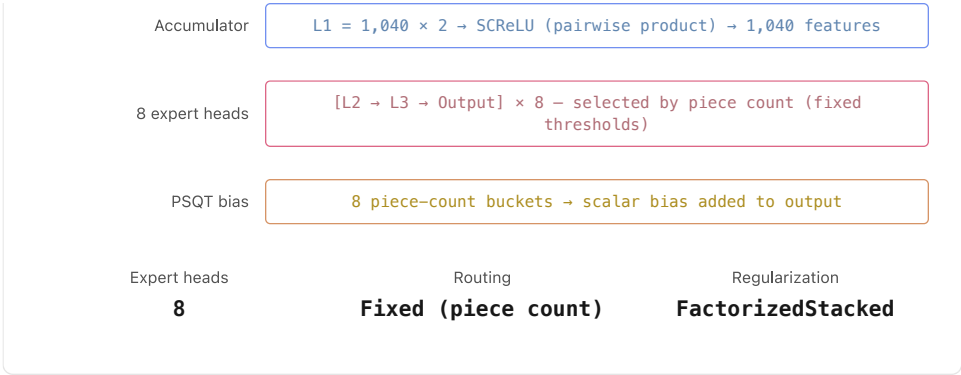
Model 2: LayerStacks NNUE

LayerStacks replaces the single stack with **8 parallel expert heads**, each a small 3-layer MLP. The correct head is selected at inference by **piece count** using fixed thresholds — no learning in the selection.

Architecture:

- Same feature encoding and accumulator as No Stack (L1 = 1,040 per perspective)
- 8 parallel expert heads: each L1 → L2 → L3 → 1
- Each head uses **FactorizedStackedLinear**: a per-bucket weight delta on top of a shared base weight. This regularizes low-data buckets toward the global mean while allowing specialization.
- Head selection: piece count → fixed bucket index (0 = fewest pieces / endgame, 7 = most pieces / opening)





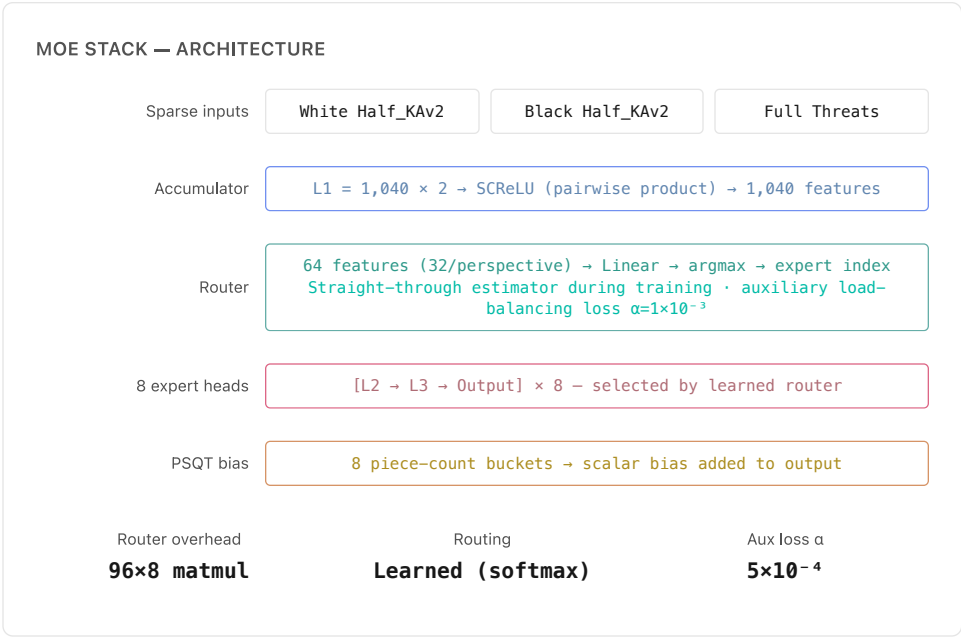
LayerStacks is the **key intermediate**: it tests whether game-phase-aware specialization (hand-coded routing) improves over a single shared network.

Model 3: Mixture-of-Experts (MoE) NNUE

MoE replaces LayerStacks' fixed routing with a **learned router**, asking whether a neural network can discover better position-to-expert assignments than the hand-coded heuristic.

Architecture:

- Same feature encoding and accumulator as LayerStacks
- **Router:** a small linear layer reads **64 dedicated accumulator features** (32 per perspective) and outputs logits over 8 experts; expert selected by argmax at inference
- The router's accumulator slice is separate from the features fed to the expert heads, avoiding gradient doubling during training
- **Straight-through estimator:** expert selection is a hard argmax but gradients flow back as if the selection were continuous
- **Auxiliary load-balancing loss** ($\alpha = 5 \times 10^{-4}$): penalizes uneven expert utilization to prevent router collapse
- **Z-loss** for further router stabilization (disabled by default)
- On GPU: sparse grouped kernel; on CPU: standard dense forward



Results and Discussion

Quantitative Metrics

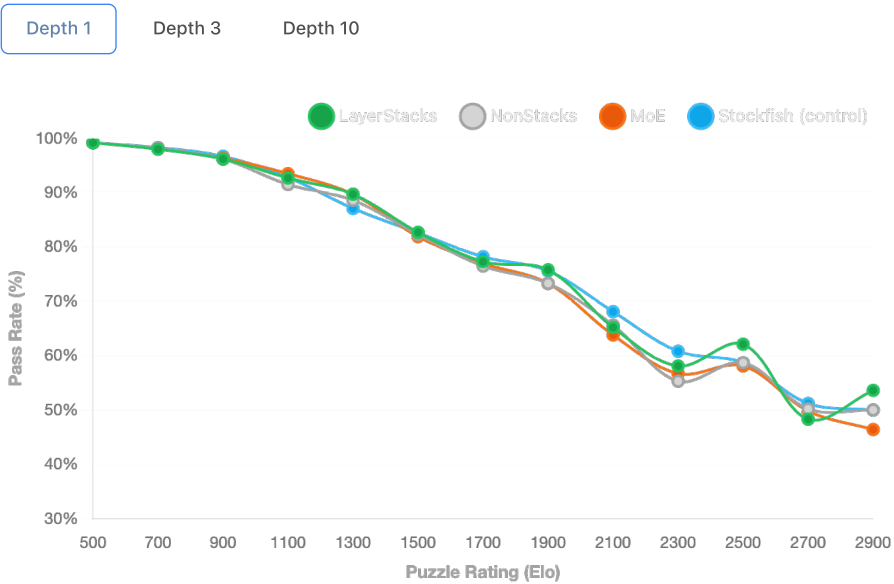
All models were evaluated in our custom Stockfish fork on the Lichess Puzzle DB. The table below reports a 10,000-puzzle sweep for ``NonStacks``, ``LayerStacks``, ``MoE`` (``RLR01-E99``), and ``Stockfish 18`` as a control; absolute ELO comes from our anchored round-robin tournament.

Metric	NonStacks	Layer Stacks	MoE	Stockfish (control)
Absolute ELO (tournament)	3722.5 ± 23.8	3729.5 ± 23.4	3711.1 ± 24.9	3825 (anchor)
Puzzle Pass Rate — depth@1	81.9%	82.7%	82.2%	82.9%
Puzzle Pass Rate — depth@3	87.3%	87.4%	87.2%	88.1%
Puzzle Pass Rate — depth@10	98.1%	98.4%	98.3%	98.4%

Puzzle evaluations use 10,000 Lichess puzzles (CC0) with 200-Elo buckets; the control column is Stockfish 18.

Puzzle Pass Rate by ELO and Depth

The chart below shows per-bucket pass rates across the puzzle ELO spectrum on the 10,000-puzzle sweep. Use the depth tabs to switch between depth@1, depth@3, and depth@10 search.



Puzzle pass rate vs. Lichess puzzle rating on 10,000 puzzles, evaluated at depths 1, 3, and 10. Displayed 200-Elo bins (500–2900) contain: 370, 603, 1238, 1446, 1268, 1074, 1108, 953, 787, 582, 327, 205, 28 puzzles.

Key observations:

- At `depth@1`, ``LayerStacks`` is the strongest learned model at **82.7%**, while ``Stockfish (control)`` is slightly higher at **82.9%** overall

- At **depth@3**, all three learned models cluster tightly (**87.2-87.4%**), with **`Stockfish (control)`** holding the overall lead at **88.1%**
- At **depth@10**, search washes out most evaluation differences: **`LayerStacks`** and **`Stockfish (control)`** tie at **98.4%**, while **`MoE`** reaches **98.3%**
- Above roughly **2300 puzzle Elo**, **`LayerStacks`** tends to hold up best among the learned models, but the far-right bins are noisy because the sample counts drop quickly

Visualizations — LayerStacks Bucket Specialization

LAYERSTACKS · PER-BUCKET WEIGHT SIMILARITY

Cosine similarity between the eight output-bucket L1 weight matrices (after coalescing the factorized component). A value near 1.0 means two buckets learned nearly the same transformation; values closer to zero mean they specialized. B0 handles positions with the fewest pieces (endgame), B7 the most (opening).

	B0 (endgame)	B1	B2	B3	B4	B5	B6	B7 (opening)
B0 (endgame)	1.00	0.73	0.75	0.75	0.75	0.75	0.74	0.74
B1	0.73	1.00	0.80	0.76	0.74	0.73	0.73	0.74
B2	0.75	0.80	1.00	0.91	0.85	0.82	0.81	0.81
B3	0.75	0.76	0.91	1.00	0.94	0.89	0.86	0.85
B4	0.75	0.74	0.85	0.94	1.00	0.94	0.91	0.88
B5	0.75	0.73	0.82	0.89	0.94	1.00	0.96	0.91
B6	0.74	0.73	0.81	0.86	0.91	0.96	1.00	0.95
B7 (opening)	0.74	0.74	0.81	0.85	0.88	0.91	0.95	1.00
	0.50				1.00			

source: checkpoints/epoch=199-step=152600.ckpt · epoch 199

The heatmap shows cosine similarity between each pair of bucket weight matrices. Key findings:

- Off-diagonal values range from 0.73–0.94, confirming that **`FactorizedStackedLinear`** regularization works
- Middlegame buckets (B2–B5) cluster tightly
- Endgame bucket (B0) diverges most; endgame requires different evaluation (king activity, pawn advancement)

Visualizations — Learned Piece-Square Tables

HALFKAV2_HM · LEARNED PIECE-SQUARE TABLES (PSQT)

OUTPUT BUCKET

Average PSQT weight (own-perspective pieces) across all 32 king buckets, one 8x8 board per piece. Each bucket corresponds to a piece-count slice; B0 is endgame, B7 is opening.

B4



P								N							
0.20 ... 0.28								1.19 ... 1.37							
0.21	0.21	0.21	0.21	0.21	0.21	0.21	0.21	1.19	1.27	1.28	1.31	1.31	1.30	1.29	1.21
0.28	0.27	0.24	0.24	0.24	0.23	0.25	0.27	1.28	1.30	1.32	1.33	1.34	1.34	1.32	1.30

0.27	0.25	0.25	0.24	0.23	0.22	0.23	0.27	1.31	1.33	1.34	1.35	1.36	1.37	1.35	1.32		
0.24	0.23	0.23	0.23	0.24	0.22	0.22	0.24	1.32	1.34	1.35	1.36	1.36	1.36	1.35	1.33		
0.21	0.23	0.23	0.22	0.24	0.23	0.22	0.23	1.31	1.33	1.34	1.35	1.36	1.36	1.34	1.32		
0.20	0.23	0.21	0.22	0.23	0.22	0.23	0.22	1.28	1.31	1.33	1.35	1.35	1.34	1.33	1.32		
0.20	0.22	0.20	0.21	0.24	0.24	0.23	0.21	1.29	1.30	1.31	1.32	1.33	1.33	1.33	1.31		
0.21	0.21	0.21	0.21	0.21	0.21	0.21	0.21	1.24	1.29	1.29	1.31	1.32	1.33	1.31	1.29		
♙ B								1.37 ... 1.43				♖ R				2.12 ... 2.20	
1.39	1.40	1.38	1.39	1.40	1.40	1.40	1.38	2.17	2.18	2.19	2.19	2.19	2.19	2.19	2.20		
1.39	1.39	1.40	1.40	1.42	1.41	1.42	1.38	2.18	2.18	2.19	2.20	2.19	2.19	2.18	2.19		
1.40	1.42	1.42	1.42	1.41	1.42	1.42	1.42	2.16	2.17	2.18	2.18	2.18	2.18	2.18	2.17		
1.39	1.41	1.42	1.42	1.42	1.41	1.42	1.41	2.16	2.17	2.18	2.18	2.17	2.17	2.16	2.16		
1.39	1.41	1.41	1.43	1.42	1.42	1.40	1.41	2.16	2.17	2.17	2.18	2.16	2.16	2.14	2.14		
1.39	1.40	1.41	1.41	1.43	1.41	1.41	1.39	2.15	2.16	2.17	2.17	2.16	2.15	2.13	2.14		
1.38	1.39	1.40	1.40	1.40	1.42	1.40	1.39	2.16	2.16	2.17	2.16	2.16	2.14	2.13	2.13		
1.38	1.38	1.38	1.39	1.40	1.40	1.40	1.37	2.15	2.16	2.15	2.16	2.15	2.14	2.12	2.12		
♚ Q								4.20 ... 4.27				♔ K				-0.01 ... 0.03	
4.24	4.25	4.26	4.25	4.24	4.25	4.25	4.24	0.00	0.00	0.00	0.00	0.00	0.01	0.00	-0.00		
4.25	4.25	4.25	4.25	4.24	4.25	4.25	4.25	0.00	0.00	0.00	0.00	-0.01	0.01	0.02	0.00		
4.25	4.25	4.25	4.25	4.25	4.24	4.24	4.24	0.00	0.00	0.00	0.00	-0.00	0.01	0.02	0.00		
4.24	4.24	4.24	4.25	4.24	4.24	4.23	4.23	0.00	0.00	0.00	0.00	0.01	0.01	0.02	-0.00		
4.25	4.26	4.25	4.24	4.24	4.24	4.23	4.22	0.00	0.00	0.00	0.00	0.02	0.03	0.03	0.01		
4.27	4.26	4.25	4.24	4.23	4.22	4.21	4.21	0.00	0.00	0.00	0.00	0.01	0.02	0.02	0.01		
4.26	4.25	4.24	4.23	4.23	4.21	4.20	4.20	0.00	0.00	0.00	0.00	0.00	-0.00	0.00	0.01		
4.26	4.25	4.24	4.23	4.23	4.21	4.20	4.20	0.00	0.00	0.00	0.00	0.00	-0.01	-0.00	-0.01		
source: checkpoints/epoch=199-step=152600.ckpt · epoch 199																	

The PSQT boards show piece values learned from data alone — no hand-coded material values.

Opening bucket:

- Knights and bishops prefer central squares, matching opening principles
- King values are near-zero (castling safety, not activity)

Middlegame bucket:

- Bishop values rise above knights — the model learns bishops outperform knights in open positions
- Pawns prefer early advancement

Endgame bucket:

- Pawn values rise on advanced ranks — the model learns passed pawn value
- King values turn positive, reflecting the endgame principle of king activity

Visualizations — MoE Expert Specialization

MOE · PER-EXPERT WEIGHT SIMILARITY

Cosine similarity between the eight expert L1 weight matrices. Unlike LayerStacks (which uses fixed piece-count routing), the MoE router learns which expert to select. Lower off-diagonal values indicate the router has discovered distinct specializations beyond simple game phase.

	E0	E1	E2	E3	E4	E5	E6	E7
E0	1.00	0.61	0.59	0.62	0.60	0.62	0.60	0.65
E1	0.61	1.00	0.59	0.55	0.53	0.55	0.55	0.61
E2	0.59	0.59	1.00	0.66	0.62	0.59	0.60	0.66
E3	0.62	0.55	0.66	1.00	0.70	0.71	0.65	0.72
E4	0.60	0.53	0.62	0.70	1.00	0.72	0.69	0.73
E5	0.62	0.55	0.59	0.71	0.72	1.00	0.75	0.76
E6	0.60	0.55	0.60	0.65	0.69	0.75	1.00	0.78
E7	0.65	0.61	0.66	0.72	0.73	0.76	0.78	1.00
	0.50					1.00		

source: moe-rlr01-e99.nnue · MoE (RLR01, epoch 99)

The MoE expert similarity heatmap reveals strikingly different structure from LayerStacks:

- **Off-diagonal values range from 0.53–0.78**, much lower than LayerStacks (0.73–0.94), confirming the router has learned meaningfully distinct experts
- **Experts E3–E7 form a loose cluster** (similarities 0.65–0.78), while **E0 and E1 diverge** from all others (~0.53–0.61)
- The pattern is **not a simple gradient** like LayerStacks' piece-count ordering — the router has discovered a non-trivial partitioning of position space

Visualizations — MoE Learned Piece-Square Tables

MOE · LEARNED PIECE-SQUARE TABLES (PSQT)

OUTPUT BUCKET

Average PSQT weight (own-perspective pieces) across all 32 king buckets. These values are from the MoE model's Feature Transformer — the PSQT output buckets are still indexed by piece count (B0 = endgame, B7 = opening), independent of the learned expert routing.

B4



P								N							
0.21 ... 0.34								1.10 ... 1.35							
0.21	0.21	0.21	0.21	0.21	0.21	0.21	0.21	1.19	1.24	1.25	1.28	1.28	1.29	1.27	1.26
0.23	0.25	0.24	0.24	0.26	0.27	0.27	0.24	1.22	1.27	1.27	1.28	1.29	1.29	1.28	1.28
0.22	0.26	0.24	0.25	0.26	0.28	0.26	0.25	1.23	1.28	1.30	1.32	1.32	1.31	1.30	1.28
0.24	0.25	0.27	0.28	0.28	0.27	0.27	0.26	1.25	1.29	1.32	1.33	1.33	1.33	1.32	1.28
0.27	0.27	0.26	0.27	0.28	0.29	0.28	0.28	1.29	1.32	1.33	1.34	1.34	1.35	1.34	1.31
0.31	0.25	0.27	0.27	0.28	0.28	0.27	0.31	1.26	1.31	1.31	1.32	1.34	1.35	1.32	1.30
0.34	0.29	0.25	0.26	0.25	0.26	0.26	0.33	1.22	1.23	1.28	1.30	1.30	1.30	1.27	1.26
0.21	0.21	0.21	0.21	0.21	0.21	0.21	0.21	1.10	1.22	1.23	1.28	1.28	1.25	1.26	1.15

♙ B								1.34 ... 1.41								♖ R								2.10 ... 2.21							
1.35	1.36	1.35	1.36	1.37	1.36	1.37	1.34	2.14	2.15	2.15	2.15	2.15	2.13	2.11	2.10																
1.36	1.36	1.37	1.38	1.37	1.40	1.38	1.38	2.16	2.16	2.17	2.17	2.16	2.14	2.12	2.11																
1.36	1.38	1.39	1.39	1.40	1.39	1.39	1.36	2.16	2.16	2.17	2.18	2.16	2.14	2.12	2.12																
1.37	1.39	1.39	1.41	1.41	1.41	1.38	1.39	2.16	2.16	2.18	2.18	2.17	2.16	2.15	2.13																
1.38	1.39	1.40	1.40	1.41	1.39	1.40	1.38	2.16	2.17	2.17	2.18	2.18	2.17	2.17	2.16																
1.38	1.41	1.40	1.40	1.39	1.40	1.40	1.40	2.17	2.18	2.18	2.18	2.19	2.19	2.18	2.17																
1.35	1.37	1.38	1.37	1.38	1.38	1.38	1.36	2.19	2.20	2.20	2.20	2.21	2.20	2.19	2.19																
1.34	1.36	1.35	1.35	1.37	1.36	1.38	1.35	2.17	2.19	2.19	2.19	2.20	2.20	2.20	2.19																
♚ Q								4.23 ... 4.32								♗ K								-0.06 ... 0.09							
4.31	4.31	4.29	4.28	4.27	4.25	4.24	4.24	0.00	-0.02	-0.02	0.01	0.02	0.03	0.01	0.02																
4.31	4.31	4.29	4.28	4.26	4.24	4.23	4.25	-0.04	-0.03	-0.03	-0.03	0.05	0.05	0.02	0.03																
4.32	4.31	4.30	4.28	4.27	4.26	4.25	4.25	-0.06	-0.06	-0.06	-0.02	0.05	0.09	0.06	0.04																
4.29	4.30	4.29	4.28	4.29	4.28	4.28	4.27	-0.02	-0.05	-0.04	-0.02	0.02	0.03	0.03	0.02																
4.29	4.29	4.29	4.29	4.29	4.28	4.27	4.27	-0.01	-0.02	-0.03	0.00	0.01	-0.00	-0.00	0.02																
4.29	4.29	4.29	4.29	4.29	4.28	4.28	4.29	0.01	-0.01	-0.00	-0.01	-0.01	-0.01	-0.01	0.00																
4.29	4.29	4.29	4.29	4.28	4.28	4.29	4.29	0.01	0.01	0.01	0.01	-0.00	-0.00	-0.01	-0.00																
4.28	4.30	4.29	4.28	4.27	4.28	4.28	4.29	0.02	-0.00	0.01	0.00	-0.01	-0.00	-0.02	0.00																
source: moe-rlr01-e99.nnue · MoE (RLR01, epoch 99)																															

The MoE model's PSQT weights share the same 8 piece-count buckets as LayerStacks (since PSQT selection is always by piece count, independent of the learned router). Compared to LayerStacks:

Similarities:

- Pawn advancement is valued in endgame buckets; central knights/bishops dominate in opening/middlegame
- King values transition from near-zero (opening) to positive (endgame), matching known chess principles

Differences:

- The MoE model shows slightly more pronounced piece-value gradients across buckets, suggesting the Feature Transformer compensates for the router's different expert allocation

Analysis and Tradeoffs

Why LayerStacks beats NonStacks

Game-phase-aware heads learn different evaluation functions for endgames vs. openings. A single hidden layer must compromise between conflicting signals (endgame king activity vs. opening development). LayerStacks avoids this via specialization, gaining +7 ELO over NonStacks and also leading the learned models on the 10,000-puzzle sweep.

Why MoE underperforms

MoE training is harder than LayerStacks. The expert weight similarity heatmap (above) shows that experts **did learn distinct transformations** — off-diagonal cosine similarities (0.53–0.78) are substantially lower than LayerStacks (0.73–0.94). On the larger 10,000-puzzle sweep, MoE is much more competitive than in our earlier smaller benchmark: it lands at **82.2% / 87.2% / 98.3%** across depths 1/3/10, very close to LayerStacks. However, that puzzle parity still does not translate into the best absolute ELO. Possible explanations:

1. **Routing instability during training** — the straight-through estimator introduces gradient noise; experts may have specialized on inconsistent subsets of positions as the router shifted
2. **Insufficient training epochs** — the MoE model (epoch 99) had less training than LayerStacks (epoch 199); the router may need longer to converge to a stable assignment
3. **Expert load imbalance** — despite auxiliary losses ($\alpha = 5 \times 10^{-4}$), some experts may have received disproportionate traffic, leaving others undertrained
4. **The fixed piece-count heuristic is a strong prior** — LayerStacks' hand-coded routing gives a near-optimal partitioning for free; the MoE router must discover an equally good (or better) partitioning from scratch

Costs of each approach

	NonStacks	LayerStacks	MoE
Inference overhead	Baseline	Same (fixed branch)	+64x8 router matmul
Serialization complexity	Simple	Moderate	High (custom Stockfish build)
Training stability	High	High	Low (router collapse risk)
Absolute ELO	3722.5	3729.5	3711.1

The 64x8 router matmul is negligible at inference. The real costs are training instability and serialization complexity.

Next Steps

1. **Retrain MoE with higher α and longer schedules** to stabilize routing and test whether the remaining puzzle gap can close the larger ELO gap
2. **Scale training data** with additional Stockfish and Leela corpora
3. **Scale accumulator** from 1,040 $\rightarrow$ 2,048 neurons per perspective
4. **Validate via tournament play** using Cute Chess at longer time controls
5. **Analyze MoE routing decisions** to understand whether the router discovers meaningful game-phase clusters

References

Silver et al. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *arXiv*. <https://arxiv.org/pdf/1712.01815>

Nasu, Y. (2018). "NNUE: Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi." *Ziosoft*. https://github.com/asdfjkl/nnue/blob/main/nnue_en.pdf

Monroe, D., & Chalmers, P. (2024). Mastering Chess with a Transformer Model. *arXiv*. <https://arxiv.org/abs/2409.12272>

Shaw et al. (2018). "Self-Attention with Relative Position Representations." *arXiv*. <https://arxiv.org/abs/1803.02155>

Lepikhin et al. (2021). "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding." *ICLR*. <https://arxiv.org/abs/2006.16668>

Fedus et al. (2022). "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity." *JMLR*. <https://arxiv.org/abs/2101.03961>

Contributions

Name	Final Contributions
Rohan Godha	MoE architecture design & implementation, custom Stockfish fork integration, training infrastructure
Akash Sarode	LayerStacks architecture, FactorizedStackedLinear, PyTorch trainer rewrite
Sai Lalith Kanumuri	Rust dataloader, feature encoding (HalfKAv2_hm + Full_Threats), position filtering
Pranav Vyas	Visualizations (PSQT, bucket heatmaps, pass rate charts), checkpoint analysis scripts
Rayan Das	Evaluation pipeline, puzzle benchmark, ELO measurement, results analysis

Gantt Chart

[View Gantt Chart \(PDF\)](#).

Gantt Chart Proposal Midterm Final Report ☐ Spring Break

Task	Owner	Jan 16	Jan 23	Jan 30	Feb 6	Feb 13	Feb 20	Feb 27
Project Team Composition	All							
PROJECT PROPOSAL								
Introduction & Background	Rohan							
Problem Definition	Rohan							
Methods	Pranav & Akash							
Potential Dataset	Pranav & Akash							
Potential Results & Discussion	Rayan & Sai							
Video Creation & Recording	All							
GitHub Page	Rohan							
MIDTERM REPORT								
M1 Design & Selection	Rohan, Pranav...							
M1 Data Cleaning	Rohan							
M1 Data Visualization	Pranav							
M1 Feature Reduction	Akash							
M1 Implementation & Coding	Rayan							
M1 Results Evaluation	All							
M2 Design & Selection	Rohan, Pranav...							
M2 Data Cleaning	Rohan							
M2 Data Visualization	Pranav							
M2 Feature Reduction	Akash							
M2 Coding & Implementation								

M2 Results Evaluation	Sai All							
Midterm Report Write-up	All							
FINAL REPORT								
M3 Design & Selection	Rohan, Pranav...							
M3 Data Cleaning	Rohan							
M3 Data Visualization	Pranav							
M3 Feature Reduction	Akash							
M3 Implementation & Coding	Rayan							
M3 Results Evaluation	All							
M1-M3 Comparison	All							
Video Creation & Recording	All							
Final Report Write-up	All							